

AudioCodec 项目报告

ely · 最近修订: 2026-04-20

ABSTRACT

本项目围绕 speech neural codec 展开, 目标是在 16 kHz 单声道语音设定下, 实现一个可训练、可评测、可追溯的神经音频编解码系统, 并在统一 benchmark 上与传统音频压缩算法比较压缩率与重建质量。项目早期从 Conv + RVQ + Decoder 的最小闭环出发, 确认了离散 token 化与波形重建链路可以工作, 但试听样例中持续出现电流音和数字伪影。随后系统升级为 SEANet + EMA RVQ, 并在当前代码库中加入 MS-STFT discriminator、feature matching、梯度 Balancer 与 Adam(0.5, 0.9) 配方。下文使用的 benchmark 结果均来自这一路线及其 num_quantizers 变体; 在当前 held-out slice 的抽样试听中, 8 / 12 kbps 点已经达到高保真重建水平, 整体效果远超起步阶段预期。

在最终系统中, 我们保持骨干和训练配方不变, 仅通过调整 RVQ stage 数量构建 2 / 4 / 8 / 12 kbps 的 neural bitrate ladder, 并将其与 Opus / MP3 / AAC / FLAC 在统一 held-out manifest 上进行比较。这里既包含更偏通用音频的传统算法, 也包含 speech-oriented 的 Opus。neural-2k 的实际码率为 2.00 kbps, 平均 STOI 为 0.886; 与之对应, 本次 benchmark 中 opus-2k、mp3-2k 和 aac-2k 的实际码率分别为 5.45、8.18 和 10.67 kbps。这些结果支持以下结论: 在当前 43 条测试语音、当前指标集合和抽样试听下, neural codec 在超低实际码率区域具有明确竞争力; 与此同时, speech-oriented 的 Opus 在 8-12 kbps 区间仍然是更强的传统基线。报告同时保留了 neural-2k 中个别样本的 timbre drift 案例, 以说明当前系统在说话人音色保真上的边界。

从课程基线到高保真 Speech Neural Codec 原型的设计、调试与 Benchmark 对比

1 Project Introduction

1.1 Problem Definition

本项目聚焦于面向语音场景的端到端神经音频编解码。具体而言, 输入被统一限制为 16 kHz、单声道 speech waveform, 系统需要学习一条从连续波形到离散 token 表示、再从离散 token 回到重建波形的完整压缩链路。项目最终并不只停留在“模型能训通”, 而是要回答一个更具体的问题: 在相同或相近的实际压缩预算下, 当前 learned codec 在 speech 任务上能达到怎样的质量-码率折中, 并与传统 codec 的 tested operating points 有何差异。

这里需要特别说明的是, 项目最初的口径中包含了“VAE + RVQ”这一表述, 但随着实现细节的落地, 最终系统更准确地说属于 RVQ autoencoder / VQ-VAE family, 而不是严格意义上的 variational autoencoder。这一表述调整并非概念修辞, 而是源于实现层面的事实: 最终系统不包含显式的 KL 正则项, 核心设计重点是离散表征、残差量化与感知型训练目标 [1]。

1.2 Working Hypothesis

本项目在实现前采用的工作假设是: 如果把任务分布限制在 speech-only 场景, learned codec 可能在低码率区间形成有竞争力的 rate-distortion operating points。这里的出发点不是把传统音频压缩算法简单理解为“设计不足”, 而是把问题看作分布约束。许多传统 codec 需要覆盖更广的音频分布, 因此必须在语音、音乐、环境声和瞬态噪声之间折中, 形成面向广义音频的压缩策略。

如果把任务范围限制在 speech 领域, 统计结构会更窄, 也更规则。说话人的发声器官、共振峰结构、语音节律、基频变化和短时频谱形态都受到较强约束, 因此 speech 分布通常比“所有音频”更容易被专门建模。基于这一点, 项目把 speech-only learned codec 是否能在低码率区间取得更好的质量-码率折中, 当作一个值得检验的经验问题。

这个假设明确边界。即使 speech-only 分布存在更高的可压缩性, 也不意味着任意 neural codec 都能在有限数据和有限优化预算下逼近这一上限。并且, 本项目的 baseline 里也包含了 speech-oriented 的 libopus -application voip 设置, 而不是只拿面向通用音频的系统作对照。因此, 本项

目把该假设作为经验检验而非理论结论：先验证 speech-only learned codec 是否能稳定训练，再通过 benchmark 观察这种优势是否在当前数据和 operating points 上出现。

1.3 Project Scope

项目的范围被刻意收紧到 speech-only、offline reconstruction 的场景。这种约束并非技术保守，而是为了保证课程周期内能够完成一个真正闭环的系统。在本次实现中，系统不追求通用音频能力，不覆盖音乐、环境声或多说话人混合场景，也不实现流式推理、低延迟端侧部署和实时系统优化。与其在多个方向上浅尝辄止，本项目优先保证以下三点同时成立：第一，模型实现和训练链路正确；第二，能够形成多码率 operating points；第三，benchmark 设计具有可追溯性和可解释性。

1.4 Why This Topic

选择 speech neural codec 作为项目主题，主要出于两个动机。第一，传统音频压缩算法虽然成熟，但其设计哲学仍然以手工设计的变换、量化和编码管线为主，而 neural codec 提供了另一种路线：直接从数据中学习压缩表示、重建路径和感知优化目标。第二，speech 场景相比音乐场景更容易在有限算力和有限时间内得到稳定可听的结果，同时又具有明确的低码率应用价值，因此非常适合作为当前实验对象。

从研究叙事角度看，这一主题还有一个额外优势：RVQ 产生的离散 token 除了用于重建，也可以被看作潜在的中间语音表示。因此，这个项目虽然以 compression 为主线，但其结果也和后续语音生成系统中常见的离散表征路线存在联系 [2]。这里不进一步验证 tokenizer 方向，只把这一点作为相关背景。

1.5 Project Deliverables

项目最终交付包含四类长期资产。第一类是模型与训练代码，包括数据加载、配置系统、神经编解码结构、量化器、损失函数和训练循环。第二类是 benchmark 与评测脚本，包括 neural codec 导出、传统 codec wrapper、统一的 manifest 和客观指标统计。第三类是实验结果本身，包括多码率 checkpoint、重建音频样例、统一 summary 表和 rate-distortion 图。第四类是文档与报告资产，包括架构说明、开发计划、benchmark runbook 和当前这份项目报告。

1.6 Development Environment

整个项目使用 Python + PyTorch + torchaudio 实现，训练主要在 Linux 服务器上进行，benchmark 则依赖 ffmpeg 提供的 libopus、libmp3lame、aac 和 flac 编码器。工程层面使用 JSON 配置驱动不同实验分支，通过 checkpoints/、metrics.jsonl、samples/ 和 evals/outputs/ 来维持实验的可追溯性。报告撰写则使用 Typst，其原因是它在保持接近 Markdown 的可写性的同时，能够提供接近 TeX 的表格与图文排版能力。

2 Technical Details

2.1 Theory Background

Neural codec 的核心思想，是把传统 codec 中原本分离的分析变换、低维表示、量化和重建过程统一到一个端到端可训练系统中。对 speech 来说，这个过程可以粗略理解为：编码器将原始波形映射为较低时间分辨率的 latent sequence，量化器把连续 latent 映射为有限离散 token，而解码器再根据这些 token 恢复波形。与传统频域变换方案相比，这种方法的优势在于表示不再被先验固定，而是由训练目标与数据分布共同决定。

在本项目中，RVQ 的作用尤其关键。单级向量量化通常难以在有限码本大小下兼顾表示能力和训练稳定性，而 residual vector quantization 通过多级逐步逼近残差的方式，把一个高保真的连续表示拆成多个低复杂度离散决策。这样做一方面使码率控制更直接，另一方面也使得系统可以在不改骨干和训练目标的情况下，仅通过调整量化 stage 数量构建一条 bitrate ladder [3]。

需要强调的是，仅有 autoencoder 和 RVQ 并不能自动带来高质量的感知重建。音频任务中的一个核心困难在于：逐点误差、线性频谱误差和人耳主观质量之间并不完全一致。因此，训练目标的

设计往往和骨干设计同样重要。这也是本项目后期把重心从“继续换模型”转向“对齐训练机制”的直接原因。

从实现角度看，本项目最终系统可以抽象为下面的四步：

1. 编码器把长度为 T 的波形 $x \in \mathbb{R}^{(1 \times T)}$ 变成长度约为 $T / 320$ 的 latent 序列 $z \in \mathbb{R}^{(128 \times L)}$ 。
2. RVQ 用多个离散 codebook 逐级逼近 z ，输出 shape 为 $[\text{num_quantizers}, L]$ 的离散索引。
3. 离散索引查表并求和得到量化后的 latent z_q 。
4. 解码器根据 z_q 重建波形 $\hat{x}$ 。

这一路径里有两个和传统 codec 很不一样的点。第一，压缩表示不是手工定义的变换域系数，而是从数据中学到的 latent manifold。第二，码率控制不是通过切换 psychoacoustic rule 或量化步长完成，而是通过离散 token 的预算直接完成。这也是为什么本项目后期可以在不改动骨干和 loss 的前提下，只靠改变 quantizer stage 数量构造一条多码率曲线。

2.2 Baseline Design and Early Simplification

项目最初从一套刻意简化的基线系统出发：1D Conv Encoder $\rightarrow$ RVQ $\rightarrow$ 1D Conv Decoder。其设计原则很明确，即优先保证实现短、变量少、可快速训练，并在课程周期内构建一个完整的 learned compression 闭环。编码器采用逐层 stride 下采样卷积与轻量残差块，解码器则使用镜像式的转置卷积恢复波形；量化器使用最朴素的 greedy residual vector quantizer；损失函数由 waveform L1、multi-scale STFT 以及量化器相关约束组成。

这一阶段最重要的贡献并不是音质，而是工程闭环。模型具备明确的 encode / decode / forward 接口，训练端完成了 deterministic split、随机裁剪、周期验证、checkpoint、试听样例导出等最小实验基础设施。更重要的是，基线很快验证了一个关键信号：系统确实能学到语音内容和时序对齐，说明“从 waveform 到 discrete token 再回 waveform”的计算图本身是通的。

然而，试听结果也立刻暴露出根本问题：尽管可以清楚听出语音内容，重建波形仍然带有明显电流音、电颤感和高频数字颗粒。这一现象具有诊断意义。它说明当前系统的瓶颈不在“训练是否收敛”，而在“当前模型和目标函数是否足以支撑可用级音质”。因此，项目后续的发展不再是简单延长基线训练，而是沿着更强骨干和更合理感知目标两个方向推进。

2.3 Model Evolution

本项目的模型演进可以概括为两次关键转折。第一次转折，是从最小 Conv + RVQ 基线转向 SEANet + EMA RVQ 的更强骨干。第二次转折，是从“仅依赖重建损失”转向“对齐 Encodec/Audiocraft 的训练目标与训练机制”。前者解决的是表达能力与量化稳定性问题，后者解决的则是感知音质与训练动力学问题。

在第一次转折中，项目引入了 Encodec-inspired 的 SEANet 风格编码器与解码器 [4]。新的骨干包含更成熟的 SConv1d / SConvTranspose1d padding 逻辑、带 dilation 的 residual block、瓶颈处的 SkipLSTM、以及 weight norm。量化器也不再是最朴素的查表式 RVQ，而升级为带 EMA 更新、k-means 初始化和 dead code replacement 的量化器，更接近 SoundStream 一类 neural codec 的实现范式 [3]。这一步直接提升了单样本 overfit 的可行性和码本稳定性，但并没有自动解决全数据训练中的高频破碎和数字沙砾问题。

第二次转折来自更系统的失败分析。项目先通过 mel loss 和 encodec-like loss reweighting 等低风险实验验证了一个判断：音质瓶颈并不只来自骨干偏弱，也来自训练目标与真实听感之间的不匹配。随后系统正式引入 MS-STFT discriminator、generator adversarial loss、feature matching、梯度 Balancer 和更接近 reference 的 optimizer 配方，整体训练制度则对齐到 Encodec 的高保真 recipe [5]。Feature matching 这类对抗式声码器训练思路也与 MelGAN 等工作一脉相承 [6]。在当前项目的实验记录里，这组改动对应到最明显的质量改善；单独更换骨干或只调整重建 loss，并没有带来同等幅度的改善。

2.4 Final Model Architecture

最终采用的主力系统是一套 SEANet + EMA RVQ speech codec。其编码器和解码器均围绕 SEANet 结构实现：输入为单声道 16 kHz 波形，配置中的 downsampling ratios 记为 [8, 5, 4, 2]，对应总 hop length 320，相应 frame rate 为 $16000 / 320 = 50$ Hz。模型 latent 维度固定为 128，瓶颈附近加入

2 层 SkipLSTM，以建模局部卷积难以覆盖的更长时程依赖。骨干宽度保持在 32 filters，这一选择兼顾了训练稳定性与算力成本。

量化器部分采用 EMAResidualVectorQuantizer。码本大小固定为 1024，因此每个 code 的名义信息量约为 10 bits。在最终 benchmark 阶段，系统保持骨干、codebook size、frame rate 和训练目标不变，仅通过调整 num_quantizers 改变码率。由于 50 Hz x 10 bits = 500 bit/s 对应一个 quantizer stage，因此系统的名义码率可以近似写为 $0.5 \times \text{num_quantizers}$ kbps。据此，项目构造了 24 / 16 / 8 / 4 stage 对应的 12 / 8 / 4 / 2 kbps neural bitrate ladder。

如果进一步展开到可复现层面，编码器、量化器和解码器的职责分工如下。

首先，编码器使用 SEANetEncoder。每一级下采样都包含两类操作：一类是带 dilation 的 residual block，用来在当前时间分辨率上扩展感受野并聚合局部上下文；另一类是 stride convolution，用来降低时间分辨率并把更多建模预算转移到 channel 维度。当前实现会根据配置中的 ratios 构造出总 hop length 320 的下采样路径，因此原始 16000 Hz 波形最终对应 50 Hz 的 latent frame rate。这一选择非常关键，因为它直接决定了“每秒生成多少帧离散 token”，也直接进入码率公式。

其次，量化器使用 EMAResidualVectorQuantizer。与单级 VQ 不同，当前实现采用逐级残差逼近。给定编码器输出 z ，第一个 codebook 先选择最接近的离散向量 q_1 ，然后对残差 $r_2 = z - q_1$ 继续量化，依次得到 $q_2, q_3, \dots, q_N$ ，最终量化表示是这些 stage 的求和 $z_q = \sum_i q_i$ 。直观上，这相当于把一个高保真连续表示分解成多个更低复杂度的离散修正。实现上，码本并不是通过普通反向传播直接更新，而是通过 exponential moving average 维护 embedding、embedding_avg 和 cluster_size，再配合首次训练批次的 k-means 初始化以及 dead code replacement 来提高稳定性。其核心优点是：在大 codebook、长训练下，码本利用率通常比直接随机初始化更稳，也更接近成熟 neural codec 的工程实现。

最后，解码器使用与编码器镜像的 SEANetDecoder。它先把量化后的 latent 沿时间轴逐步上采样，再经过多层 residual block 和最终卷积恢复波形。由于编码器与解码器共享同样的 hop length，且当前系统是离线非流式设定，因此整个路径可以稳定地实现“任意长度 utterance -> token sequence -> reconstruction”的一一对应。也正因此，benchmark 阶段的 neural codec 导出脚本并不依赖训练时的随机裁剪逻辑，而是可以直接按 utterance 整句导出重建结果。

如果只保留最小复现信息，这套系统的核心超参数其实可以压缩成下面四个：sample_rate = 16 kHz、hop_length = 320、latent_dim = 128、codebook_size = 1024。其余多码率 operating points 都只是在此基础上改变 num_quantizers。这也是本项目认为“码率变化应尽量与骨干/训练目标变化解耦”的主要原因。

Component	Setting
Backbone	SEANet encoder / decoder with dilated residual blocks and 2-layer SkipLSTM
Sample rate	16 kHz mono speech
Downsampling ratios	[8, 5, 4, 2], giving hop length 320 and frame rate 50 Hz
Latent dimension	128
Quantizer	EMA RVQ with codebook size 1024, k-means init, dead code replacement
12 kbps operating point	24 quantizers
8 kbps operating point	16 quantizers
4 kbps operating point	8 quantizers
2 kbps operating point	4 quantizers

表 1 Final neural codec architecture and bitrate ladder.

2.5 Training Objective Evolution

训练目标的演进，是本项目中最重要的调试线索之一。第一版基线使用的是 waveform L1、multi-scale STFT 和量化器的 commitment/codebook loss。这种设计能够快速建立基本可训练性，但它默认了一个较强假设：只要时域和线性频域误差足够小，听感也会同步改善。后续实验表明，这个假设在当前 speech codec 任务上并不充分。

接下来引入的 mel loss 和 encodec-like loss reweighting，属于中间过渡阶段。它们的作用不是直接形成最终配方，而是帮助项目确认目标函数会显著影响收敛到的声音形态。尤其是在保持骨干不

变的单样本 overfit 验机中，弱化 waveform 项、强化多尺度频谱项通常会更快改善试听样例，表明原始目标函数与需要优化的 perceptual quality 存在偏差。

最终用于 benchmark 的配方由 adversarial 路径和梯度配平共同组成。当前系统的 generator 端包含 waveform 重建项、multi-scale STFT 重建项、adversarial generator loss 和 feature matching loss；判别器使用五个 STFT 尺度的频谱域判别器，generator/discriminator 优化器均采用 Adam(beta1=0.5, beta2=0.9)。从当前代码实现看，量化器的 commitment loss 单独回传，codebook_weight 设为 0.0；其余围绕 output.reconstruction 的训练分量再交给本地 Balancer 做梯度重缩放，而不是直接按固定系数相加后统一反传。

为了把这一路径写得更准确，这里的权重应理解为 Balancer 的目标梯度比例，而不是一次标量求和后的唯一 backward 系数。在当前实现里，waveform_loss、stft_loss、generator_adversarial_loss 和 feature_matching_loss 的目标比例分别为 0.1 / 2.0 / 4.0 / 4.0；Balancer 会先分别计算这些分量对 output.reconstruction 的梯度范数，维护对应的 EMA norm，再按目标比例重缩放后回传。量化器的 commitment_loss 则按 commitment_weight = 1.0 单独回传，不进入这一步的梯度配平。

这一点也是 Balancer 在报告里需要单独说明的原因。如果简单用固定系数把这些 loss 直接相加，某个梯度量级过大的项就可能主导整个更新，进而导致静音塌缩或 feature matching 发散。当前实现选择在共享张量上显式配平梯度贡献，这也是后续成功 run 与早期失败实验之间的重要区别。

还有一个对复现很重要的点是 optimizer 语义。最终成功实验使用的是 Adam 而不是 AdamW，并采用 (beta1, beta2) = (0.5, 0.9)、weight_decay = 0。这看起来像细节，但在对抗训练里属于 recipe 的组成部分，因此这个设置值得在报告里明确写出。

Training element	Final setting
Waveform loss	weight 0.1
Multi-scale STFT loss	weight 2.0 with FFT sizes 512 / 1024 / 2048
Mel loss	disabled in final high-fidelity recipe
Adversarial loss	enabled, weight 4.0, hinge objective
Feature matching	enabled, weight 4.0
Quantizer losses	commitment weight 1.0, codebook weight 0.0
Discriminator	MS-STFT discriminator with five STFT scales: 1024 / 2048 / 512 / 256 / 128
Gradient balancing	Balancer rescales waveform / STFT / adversarial / feature-matching gradients by EMA norms
Generator optimizer	Adam, learning rate 3e-4, betas (0.5, 0.9), no weight decay
Discriminator optimizer	Adam, learning rate 2e-4, betas (0.5, 0.9), no weight decay
Training clips	2.0 s clips, batch size 8

表 2 Final training recipe that produced the reportable neural codec results.

2.6 Key Debugging and Engineering Lessons

项目开发过程中最重要的工程经验，并不是某条特定配置，而是诊断顺序。首先，single-example overfit 被证明是整个训练链路的必要闸门。多次关键决策都依赖于这一验机实验：如果单样本都学不住，优先怀疑实现错误或训练语义；如果单样本能学住但全数据仍差，则问题更可能在泛化、目标函数或优化机制。项目最终的对抗训练修复，也是通过 overfit 路线先验证后才推广到全数据训练。

其次，音频 codec 里“数值指标改善”并不必然等于“听感改善”。本项目早期多次遇到这种情况：共享频谱指标有所下降，但试听中仍有明显电流音、数字沙砾感或声线变化。因此，试听样例并不是客观指标之外的补充展示，而是调试过程中的核心证据。尤其在 2 kbps 等极低码率点上，STOI 可以仍然很高，但 timbre 已经开始漂移；这类现象只能通过人工试听和 case study 结合理解。

第三，对抗训练的失败模式必须被当作算法问题而不是“训练波动”。项目实际踩到过静音塌缩、feature matching 发散、优化器语义不对、resume 状态不完整等问题。后期之所以能逐步纠正这些问题，很大程度上得益于配置开关、实验隔离和显式归档，使得每一轮改动都可以被定位为“哪个组件改变了什么”，而不是在同一配置上不断覆盖历史。

2.7 Bitrate Ladder Design

在最终 benchmark 设计中，项目刻意避免了“每个码率都重新设计一套模型”的做法，而是固定成功的神经编解码 recipe，仅通过 num_quantizers 控制压缩预算。这种设计有两个优点。第一，它让 rate-distortion 曲线的解释更干净，因为不同 operating point 之间的性能变化主要反映离散预算变化，而不混入骨干结构或训练目标变化。第二，它使 neural codec 的实际码率几乎精确可控，这一点在传统 codec baseline 上反而并不天然成立。

具体来说，在当前配置下，一个量化 stage 的 nominal budget 约为 $50 \text{ Hz} \times \log_2(1024) = 500 \text{ bit/s}$ ，因此 4 / 8 / 16 / 24 个 quantizers 分别对应 2 / 4 / 8 / 12 kbps。这使得不同 neural operating points 之间的主要变化来自离散预算，而不是骨干或训练目标的变化。抽样试听显示四个点都能保留语音内容，但 2 kbps 的个别样本会出现 timbre drift 和说话人声线变化。

Model	Quantizers	Nominal kbps	Best / logged steps	Observation
Neural-12k	24	12.0	91000 / 100000	lowest logged val loss 2.176
Neural-8k	16	8.0	98000 / 100000	lowest logged val loss 2.208
Neural-4k	8	4.0	148000 / 150000	lowest logged val loss 2.162
Neural-2k	4	2.0	157000 / 180000	lowest logged val loss 2.260

表 3 Final neural bitrate ladder and benchmark checkpoints. The second step count is the last validation step recorded in each run directory's metrics.jsonl.

2.8 Benchmark Pipeline Design

为了避免训练代码和评测代码耦合，最终 benchmark 被单独组织在 evals/ 目录下。整个流程分为四步：先通过 deterministic split 构建统一测试 manifest；然后导出四个 neural codec operating point 的重建结果；接着调用 ffmpeg wrapper 运行 Opus / MP3 / AAC / FLAC；最后统一计算 actual_bitrate_kbps、compression_ratio_vs_pcm16、STOI、LSD、multi_scale_stft 和 SI-SDR 等指标。这种设计的目的，是保证所有 codec 都在完全相同的 utterance 集合上比较，并且所有结果都可以追溯到统一 manifest。

benchmark 方法学里一个非常关键的原则，是不把“目标码率”直接当作最终比较坐标。对 neural codec 来说，离散 token 预算由 $\text{frame_rate} \times \text{num_quantizers} \times \log_2(\text{codebook_size})$ 决定，因此实际码率几乎可以精确控制；但传统 lossy codec 的 target bitrate 更像 operating point 输入，而不是必须精确命中的数学约束。基于这一点，下文的主结论一律建立在 actual bitrate 和真实压缩字节数上，而不是“向编码器请求了多少 kbps”。

从实现上看，benchmark 的数据与结果组织也是刻意设计过的。第一步，build_manifest.py 根据训练配置中的 deterministic split 规则生成 benchmark manifest，从而避免“训练集与测试集定义不一致”或“人工挑样本”带来的歧义。第二步，export_neural_codec.py 使用 checkpoint 逐条导出 neural reconstructions，并根据 $\text{num_frames} \times \text{num_quantizers} \times \log_2(\text{codebook_size})$ 统计 neural payload bytes，而不是错误地拿 checkpoint 大小充当压缩成本。第三步，run_traditional_codec.py 只负责把统一 manifest 中的音频交给 ffmpeg 编码/解码，因此所有传统 codec 都走同一套输入输出接口。第四步，score_outputs.py 在统一的文件级 manifest 上计算指标并汇总成 summary.csv 与 per_file_metrics.jsonl，为下文的表格、图和 case study 提供同一数据源。

3 Experiment Results

3.1 Experimental Questions

本项目实验部分主要回答三个问题。第一，在同一神经编解码框架下，是否可以构建一条从 2 kbps 到 12 kbps 的有效 speech bitrate ladder。第二，在真正低码率的 operating points 上，neural codec

与传统 codec 相比是否具有更好的质量-码率折中。第三，系统在极低码率下的主要退化形式是什么：是语音内容丢失、背景噪声增强，还是 timbre/speaker identity 的漂移。

3.2 Training Validation

训练验证可以分成两个层面。第一个层面是单样本 overfit。debug-overfit-adversarial-msstft-balanced 的验证总损失从 step 1000 的 2.56 下降到 step 4000 的 1.15，对应的 waveform loss 从 0.0077 下降到 0.0020，STFT 项则从 1.28 降到 0.57。这些日志至少说明 adversarial + balanced 训练链路可以稳定拟合同一个样本。与之配套导出的试听样例也显示 step 4000 相比 step 1000 更自然，但这部分只作为调试证据，不构成正式感知结论。

第二个层面是全数据训练。四条 neural ladder 在各自 run 目录的 metrics.jsonl 中，最佳验证步数分别为 91k / 100k、98k / 100k、148k / 150k 和 157k / 180k，其中前一个数字是最佳 checkpoint step，后一个数字是该 run 中最后一次记录到的验证步数。就本项目这四条成功 run 而言，较低码率点的最佳 checkpoint 确实出现在更靠后的训练阶段，但这只应被理解为当前实验设置下的观察，而不是一般规律。以 12 kbps 路线为例，验证总损失从 step 1000 的 5.00 下降到 step 40000 的 2.33，再下降到 step 91000 的 2.18，说明继续训练仍然带来增益，只是后期改善幅度变小。

3.3 Benchmark Setup

benchmark 使用当前仓库定义的 deterministic held-out split，总计 43 条 speech utterances，时长约 10.07 分钟，采样率统一为 16 kHz 单声道。需要明确的是，这里的 test 不是 LibriSpeech 官方 test-clean，而是从 clean/train.100 目录按项目配置切出的 repo-local held-out slice。当前 test.jsonl 中的 43 条样本都来自同一个 speaker-chapter (1069-133709)，因此它更接近一个很窄的 held-out slice，而不是覆盖多说话人的评测集。另一方面，当前 benchmark 的打分确实来自项目配置定义的 held-out test partition，而不是训练分片；因此这些结果至少可以被解释为模型对当前 held-out utterance slice 的真实泛化，而不是对训练样本的直接记忆。更强的跨说话人泛化结论，则需要更大也更多样性的测试集才能支撑。神经编解码器使用四个训练完成的 operating points: Neural-12k、Neural-8k、Neural-4k 和 Neural-2k。传统 baseline 则包括 Opus、MP3、AAC 和无损 FLAC。其中 FLAC 仅作为无损锚点，用于验证评测管线是否存在读写或对齐问题，而不参与低码率 lossy 结论。

评价指标分为三类。压缩成本指标包括 compressed_bytes、actual_bitrate_kbps 和 compression_ratio_vs_pcm16；内容和感知相关指标包括 STOI、LSD 和 multi_scale_stft；波形一致性指标则以 SI-SDR 为补充。需要强调的是，本项目不把单一指标当作最终判据。尤其在极低码率区域，STOI 能说明 intelligibility，LSD 和 multi_scale_stft 更能反映频谱形态误差，而 SI-SDR 在 timbre drift case 上往往会更敏感。因此，最终结论依赖于多指标与试听结果的综合判断。

3.4 Objective Results

从 summary.csv 的整体结果看，当前 benchmark 确实形成了一条可解释的 neural rate-distortion ladder。四个神经 operating points 的实际码率分别为 12.01、8.00、4.00 和 2.00 kbps，与 nominal setting 非常接近。这与 neural payload 直接由 token 数量统计的实现方式一致，也说明在当前 pipeline 下 neural codec 的压缩预算比较可控。需要同时说明，这里的 neural compressed_bytes 是 export_neural_codec.py 统计的 RVQ payload bytes，不包含熵编码或文件封装开销；而传统 codec 的 compressed_bytes 则是 ffmpeg 输出文件的实际磁盘大小。

与之相对，传统 codec 在超低目标码率区间出现了明显的 target-actual 脱钩。尤其在 2 kbps 目标点，Opus、MP3 和 AAC 的实际码率分别为 5.45、8.18 和 10.67 kbps。因此，这些 ffmpeg operating points 并没有真正落在 2 kbps 附近，而是停在各自实现允许的更高码率 floor。在当前 benchmark 解读里，这意味着传统 codec 的低码率 sweep 必须按实际输出大小解释，而不能只按请求的 target bitrate 解释。

进一步看不同传统 codec 的低码率 sweep，可以看到这一现象在当前 benchmark 中重复出现。MP3 在 2 / 4 / 8 kbps 三个目标点上都落在 8.18 kbps；AAC 的 2 / 4 / 8 kbps 三个目标点都集中在 10.67-10.98 kbps；Opus 则在 2 / 4 kbps 上共同落在 5.45 kbps。因此，至少对这一组 ffmpeg 设置和当前测试集而言，actual bitrate 比 target bitrate 更适合作为统一横轴。

作为 sanity check，FLAC 锚点结果也证明评测管线本身是可信的。无损重建对应的 LSD 和 multi_scale_stft 都为 0，STOI 接近 1.0，这意味着 manifest、重采样、文件读写和指标实现之间没

有出现破坏性偏差。因此，后续不同 codec 之间的差异可以较放心地解释为压缩策略差异，而不是评测链路错误。

Codec label	Target point	Actual bitrate (kbps)
Neural-2k	2 kbps	2.00
Opus-2k	2 kbps	5.45
MP3-2k	2 kbps	8.18
AAC-2k	2 kbps	10.67

表 4 Target bitrate and actual bitrate diverge strongly for traditional codecs in the ultra-low-rate regime.

在真正低码率区域，Neural-2k 是最值得单独讨论的点。它的平均 STOI 为 0.886，LSD 为 10.08，multi_scale_stft 为 1.022。在本次 benchmark 里，与它最接近的 speech-oriented 传统对手是 Opus-2k，但该点实际落在 5.45 kbps，且 STOI 只有 0.747，LSD 和 multi_scale_stft 也更差。因此，更准确也更符合直觉的表述是：在本次测试到的 operating points 里，neural codec 在超低实际码率区域展现出了非常强、甚至有些出人意料的竞争力。与此同时，这里的 neural 码率仍应理解为 payload bitrate，而不是完整 bitstream 文件大小。

中等码率区域的结果更细。Neural-8k 相比 MP3 和 AAC 的低码率 sweep，在 LSD 和 multi_scale_stft 上更好；但与 Opus-8k 相比，则呈现出指标分裂：Opus-8k 的 STOI 更高，而 Neural-8k 的 LSD 和 multi_scale_stft 更低。到 12 kbps 左右时，Opus-12k 在当前三个客观指标上都优于 Neural-12k。换句话说，本项目并不只是在更偏通用音频的传统 codec 上取胜，也和 speech-oriented 的 Opus 做了正面对照；结论是 neural codec 在超低实际码率区间表现突出，而 Opus 在 8–12 kbps 区间仍然是更强的传统基线。抽样试听中，Neural-8k 往往比 MP3 和 AAC 的低码率点更自然；在当前 held-out slice 上，8 / 12 kbps neural 点也已经达到高保真重建水平，但这部分仍然只基于非正式试听。

Codec	Actual kbps	STOI	LSD	MS-STFT
Neural-2k	2.00	0.886	10.08	1.022
Opus-2k	5.45	0.747	27.17	2.664
MP3-2k	8.18	0.865	37.88	3.979
AAC-2k	10.67	0.866	33.79	3.475

表 5 Ultra-low-rate comparison. Among the tested points, Neural-2k is the only system that actually lands near 2 kbps; its average STOI on the held-out set is 0.886.

如果只看离散 operating points 中的 STOI，一个保守的比较方式是观察哪些传统点首次超过 Neural-2k 的 STOI = 0.886。在当前表里，对应的点分别是 Opus-8k (8.30 kbps)、AAC-12k (13.29 kbps) 和 MP3-16k (16.23 kbps)。由于这里没有做插值，也没有把离散 sweep 拟合成连续曲线，正文只把这些结果当作观测到的对照点，不把它们写成精确的“同质量所需码率”结论。

Codec	Target	Actual kbps	Comp. ratio	STOI	LSD	MS-STFT
Neural-12k	12 kbps	12.01	21.32x	0.923	9.52	0.943
Neural-8k	8 kbps	8.00	31.98x	0.918	9.58	0.953
Neural-4k	4 kbps	4.00	63.97x	0.908	9.67	0.965
Neural-2k	2 kbps	2.00	127.93x	0.886	10.08	1.022
Opus-12k	12 kbps	12.34	20.76x	0.972	7.88	0.743
Opus-8k	8 kbps	8.30	30.85x	0.956	26.99	2.496
Opus-4k	4 kbps	5.45	47.00x	0.747	27.17	2.664
MP3-16k	16 kbps	16.23	15.77x	0.961	27.49	2.468
MP3-2k	2 kbps	8.18	31.29x	0.865	37.88	3.979
AAC-16k	16 kbps	17.27	14.82x	0.967	30.73	2.881
AAC-2k	2 kbps	10.67	24.01x	0.866	33.79	3.475
MP3-default	-	24.29	10.54x	0.987	22.07	1.777
Opus-default	-	63.06	4.07x	0.999	4.24	0.290
AAC-default	-	74.99	3.42x	1.000	2.94	0.179
FLAC	-	145.88	1.76x	1.000	0.00	0.000

表 6 Representative benchmark summary used in the main text. The full benchmark table is moved to Appendix A, and all conclusions are derived from actual bitrate rather than target bitrate alone.

除了 target sweep, 本项目还额外保留了 default mode, 用于观察当前 ffmpeg encoder 默认参数会落到什么实际 operating point。当前 benchmark 中, 三类传统 codec 的 default mode 分别落在 24.29、63.06 和 74.99 kbps。抽样试听中, 这些点的失真通常比超低码率点少, 但它们并不是 bitrate-matched 对照, 因此只用于展示当前默认设置对应的 operating point, 而不直接参与低码率结论。

Codec default	Actual kbps	Compression ratio	STOI	LSD
MP3 default	24.29	10.54 x	0.987	22.07
Opus default	63.06	4.07 x	0.999	4.24
AAC default	74.99	3.42 x	1.000	2.94
FLAC	145.88	1.76 x	1.000	0.00

表 7 ffmpeg default-encoder operating points. These settings land at substantially higher actual bitrates than the low-rate neural ladder.

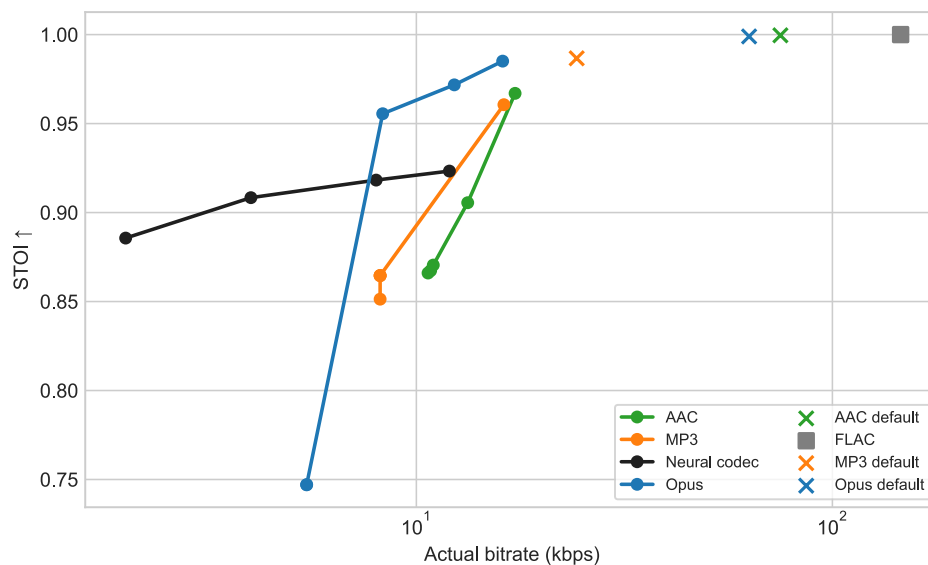


图 1 Rate-distortion view using STOI. Neural codec forms a controllable low-rate ladder, while traditional codecs occupy different actual operating points.

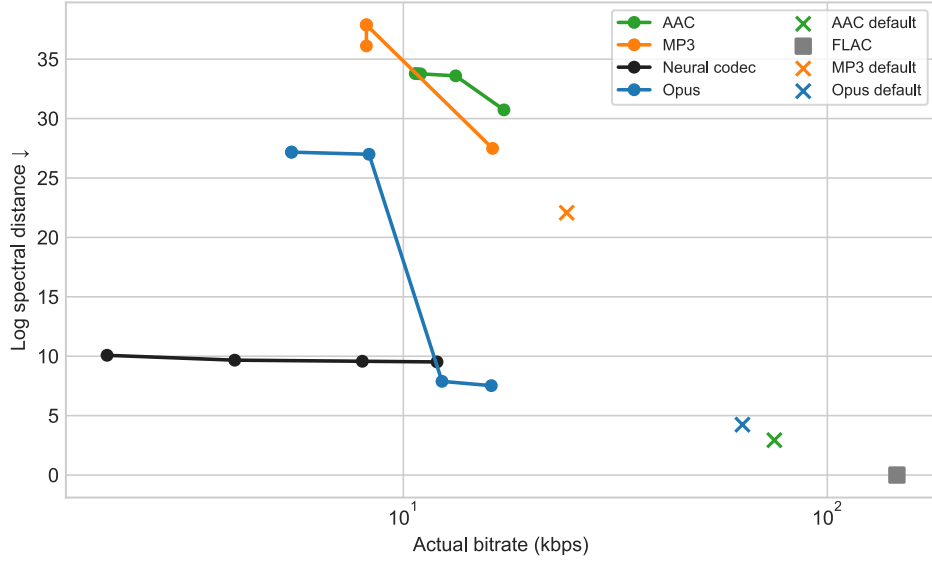


图 2 Rate-distortion view using log spectral distance. On the tested low-rate points, the neural ladder outperforms MP3 and AAC, while Opus remains the strongest classical speech baseline.

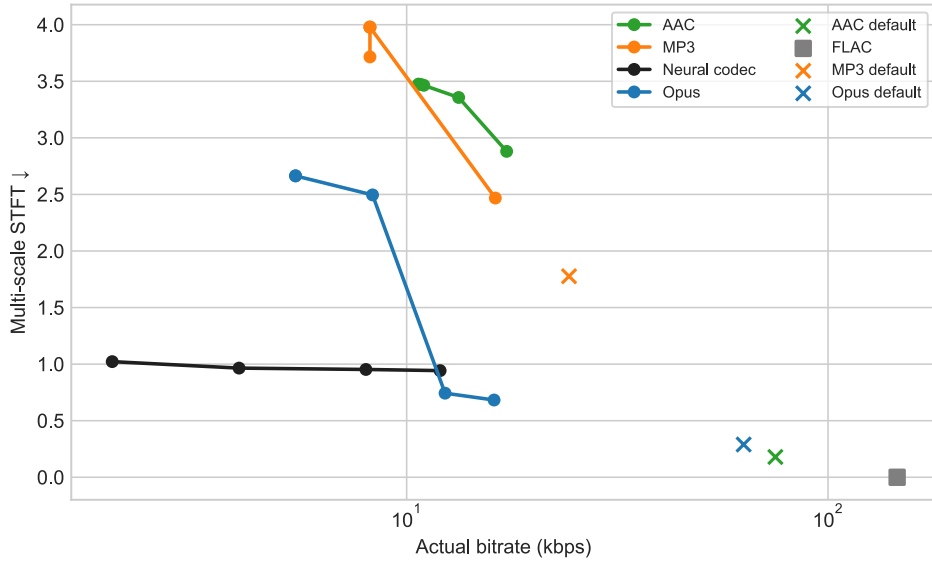


图 3 Rate-distortion view using multi-scale STFT distance. The neural ladder remains compact and predictable, whereas classical codecs show codec-specific rate-control floors at low target bitrates.

3.5 Subjective Listening Findings

主观部分只基于当前实验中的抽样试听，不构成正式听测。整体上，四个 neural operating points 都能稳定保留语音内容；12 kbps 和 8 kbps 的样例通常比 4 kbps 和 2 kbps 更自然。在当前 held-out slice 的抽样试听中，12 kbps 和 8 kbps 已达到高保真重建水平，最佳样例已经接近透明；2 kbps 的主要问题则集中在音色和细节而不是内容缺失。

在 benchmark 的抽样试听中，传统 codec 的 default operating point 通常比其低目标码率点更少出现明显 artifact，这与它们更高的实际码率一致：mp3-default、opus-default 和 aac-default 的实际码率分别约为 24.29、63.06 和 74.99 kbps。因此，default mode 的意义主要是说明当前 ffmpeg 默认设置所处的 bit budget，而不是拿来与 2-12 kbps 的 neural ladder 做同码率比较。

对 neural codec 而言，最典型的主观问题出现在 Neural-2k 的个别样本上。例如 1069-133709-0005 与 1069-133709-0027 在抽样试听中都出现了后半段声线变化。它们的共同特点是：STOI 仍然保持在 0.87 左右，说明内容与清晰度并未明显丢失；但 SI-SDR 较低，且 LSD 与 multi_scale_stft 高于

neural-2k 的总体均值。这一组合更接近“内容保住但 timbre 失真”的失败模式，也说明 2 kbps 的结论需要同时写出压缩效率和音色保真边界。

3.5.1 Case Study: Timbre Drift at 2 kbps

为了把 neural-2k 的失败模式写清楚，仅靠均值表是不够的。这里选取两个代表性样本，把“内容基本正确但说话人音色发生偏移”这件事单独展开。它们来自抽样试听中被反复提到的 utterance，同时也位于 neural-2k 单文件指标分布的较差一侧。

Sample ID	STOI	SI-SDR	LSD	MS-STFT
1069-133709-0005	0.878	0.089	10.62	1.080
1069-133709-0027	0.872	1.755	10.80	1.125

表 8 Representative neural-2k failure cases. These utterances keep relatively high STOI, but their waveform and spectral errors are already elevated enough to coincide with audible timbre drift in informal listening.

这两个样本的共同点是：STOI 仍然处在较高区间，因此模型在内容层面没有明显崩坏；但较低的 SI-SDR 和更高的频谱误差与抽样试听中的声线变化同时出现。当前证据支持把这一现象描述为本次 benchmark 中 2 kbps 的典型 artifact，但不足以把它提升为所有低码率 neural codec 的一般失败规律。

为了避免整句频谱平均后把局部差异抹平，最终 case study 图不再展示整句频谱，而是只截取局部窗口，并显式绘制 source / reconstruction / absolute log-spectral difference 三联图。局部分析也支持这种做法：在漂移样本 1069-133709-0005 的 6–7 s 区间，source 与 reconstruction 的波形相关系数为 0.263，局部 LSD 约为 0.690；而稳定对照样本 1069-133709-0006 在相同时间窗口上的相关系数约为 0.891，局部 LSD 约为 0.482。这些数字至少说明 drift 样本包含更强的局部频谱偏差。当前证据足以把这种图作为诊断工具；进一步说，一个带有启发性的猜想是，类似 absolute log-spectral difference 的局部表征可能更接近 timbre drift 所对应的误差模式。这个猜想尚未在本项目中被验证，但它确实提供了后续研究的方向。

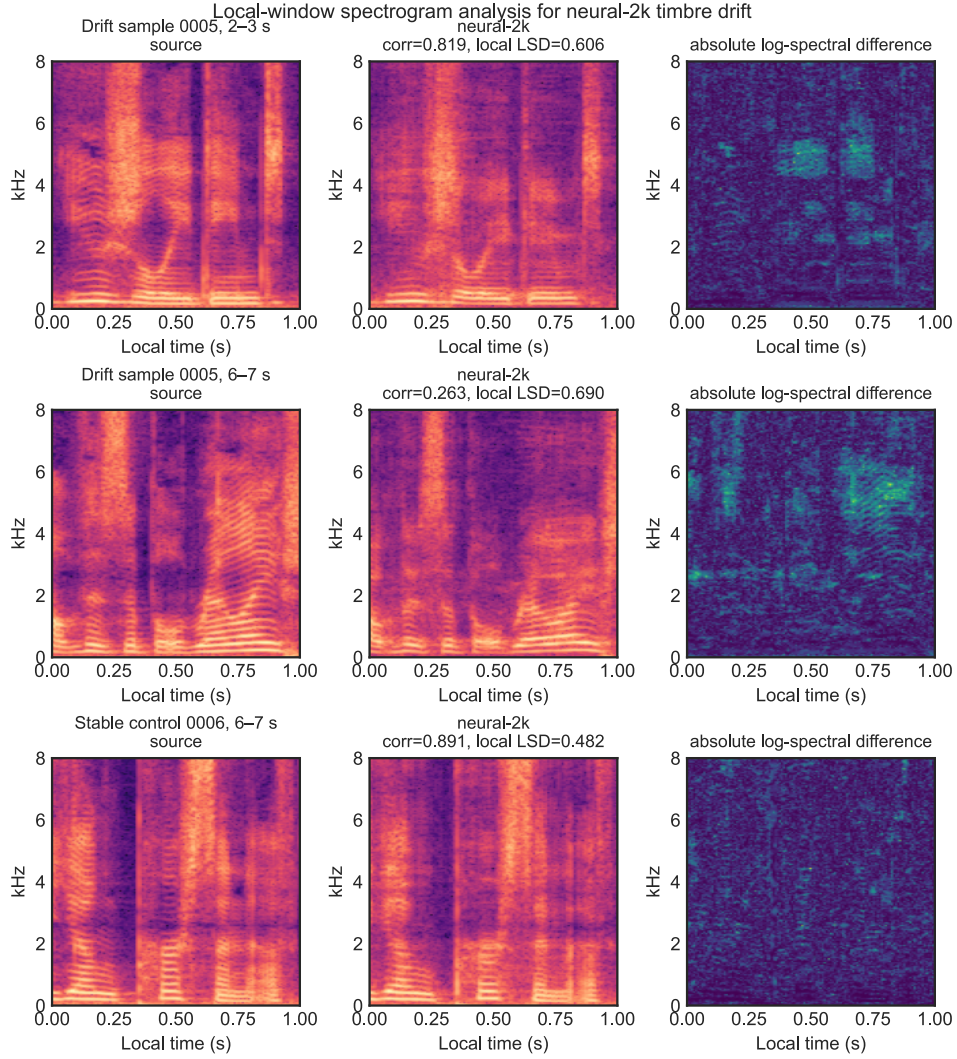


图 4 Local-window case-study figure for neural-2k. The first two rows visualize the drift sample 1069-133709-0005 in two local regions; the third row provides a stable control sample 1069-133709-0006. The rightmost column localizes absolute log-spectral difference.

3.6 Rate-Distortion Interpretation

如果把当前结果放到同一个 rate-distortion 视角下，项目的主要结论可以概括为三点。第一，当前 neural codec 的优势主要体现在最小实际码率点，而不是“在所有码率都优于传统 codec”；这一点在 Neural-2k 上最明显。第二，随着实际码率上升到 8-12 kbps，传统 codec 尤其是 Opus 会迅速变强，因此比较必须回到具体 operating point 和具体指标。第三，在当前 ffmpeg 设置下，传统 codec 的 low-target sweep 暴露出明显的 rate-control floor，这使得 actual bitrate 成为比 target bitrate 更稳定的 benchmark 坐标。

从当前工作的规模和资源约束看，这些结论已经足够强：系统不仅构建出一条多码率 learned speech codec 曲线，而且在统一 manifest 与统一 scoring 下给出了可复查、且明显强于起步预期的结果。但这些结论仍然受测试集规模、离散 operating points 和非正式听测范围的限制。

3.7 Failure Cases and Limitations

尽管结果总体可用，本项目仍然存在几个明确限制。第一，当前 benchmark 的数据规模仍然有限，测试集只有 43 条 speech utterances，而且它们都来自同一个 speaker-chapter (1069-133709)。这说明当前结果建立在一个很窄的 repo-local held-out slice 上：它足以支持本文关于当前 held-out slice 的 benchmark 结论，但还不足以推为大规模、跨说话人的统计结论。第二，主观听测仍然是轻量级的人工试听，而非正式的 MUSHRA 或更大规模双盲感知实验。第三，指标集合虽然覆盖了压缩

成本、可懂度和频谱误差，但尚未纳入 ViSQOL 等更强的感知指标，这使得高保真区域的结论仍然更依赖试听。第四，neural codec 的 compressed_bytes 来自 RVQ payload 统计，而传统 codec 的 compressed_bytes 来自 ffmpeg 输出文件大小，因此两者更适合被理解为当前实现下的压缩预算对照，而不是已经包含完整封装开销的最终部署码率。

就模型本身而言，Neural-2k 的 timbre drift 已经说明：在极限低码率下，系统保住了内容，但说话人音色还不能稳定保真。另一方面，Opus 在 8-12 kbps 区间仍然很强，说明当前系统虽然展示了神经方法的优势区间，但还没有达到可以在所有 tested rates 上压过传统 codec 的程度。更合适的定位是：这是一个可复现的研究原型，它在低码率区域展示了 learned speech codec 的潜力，也明确暴露了下一步验证工作所在的边界。

3.8 Final Conclusion

本项目最终完成了三个层面的目标。第一，在工程上，系统从零构建了一条完整的 speech neural codec 链路，覆盖模型、配置、训练、试听、benchmark 和报告资产。第二，在方法上，项目把质量改善与更具体的实现因素对应起来：从当前代码和日志看，骨干升级本身并不足够，真正与最终效果同时出现的是 SEANet + EMA RVQ、对抗训练、feature matching 和 Balancer 的组合。第三，在实验上，项目通过 2 / 4 / 8 / 12 kbps 的 neural bitrate ladder 和与 Opus / MP3 / AAC / FLAC 的统一 benchmark，给出了当前设置下的 rate-distortion 对照结果。

因此，这个项目最重要的结论不是“神经方法已经普遍优于传统 codec”，而是更具体的判断：在当前 43 条 held-out utterances、当前指标集合和当前 ffmpeg 设置下，neural codec 在超低实际码率区域表现出明确竞争力；在同一 held-out slice 的抽样试听中，8 / 12 kbps neural 点已经达到高保真重建；与此同时，speech-oriented 的 Opus 仍然在更高 tested rates 上保持明显优势。

3.9 Future Work

如果继续推进，后续工作应优先补当前结果已经暴露出的缺口，同时保留对新想法的探索空间。最直接的三项工作是：补正式听测和更强的感知指标；扩大 held-out split 并报告更稳定的跨样本统计；继续为 neural-2k 的 timbre drift 建立更系统的诊断样例。基于当前 case study，一个值得继续头脑风暴的方向是：把更局部、更偏频谱差异结构的表征发展成 timbre-oriented loss 或辅助判别信号。这个想法目前还只是猜想，不应和前文已经验证的结论混写，但它确实是本项目带出的一个研究启发。至于是否进一步引入熵模型、更多数据或更强时序建模，应当在这些基础验证完成后再决定。

4 Appendix A: Full Benchmark Table

为了让正文保持可读性，主结果部分只保留了最有解释力的 operating points。本附录则给出完整 benchmark 长表，以便读者核对所有 target sweep、default operating point 与 neural bitrate ladder 的原始统计值。

Codec	Target	Actual kbps	Comp. ratio	STOI	LSD	MS-STFT
Neural-12k	12 kbps	12.01	21.32x	0.923	9.52	0.943
Neural-8k	8 kbps	8.00	31.98x	0.918	9.58	0.953
Neural-4k	4 kbps	4.00	63.97x	0.908	9.67	0.965
Neural-2k	2 kbps	2.00	127.93x	0.886	10.08	1.022
Opus-16k	16 kbps	16.12	15.88x	0.985	7.52	0.683
Opus-12k	12 kbps	12.34	20.76x	0.972	7.88	0.743
Opus-8k	8 kbps	8.30	30.85x	0.956	26.99	2.496
Opus-4k	4 kbps	5.45	47.00x	0.747	27.17	2.664
Opus-2k	2 kbps	5.45	47.00x	0.747	27.17	2.664
Opus-default	-	63.06	4.07x	0.999	4.24	0.290
MP3-16k	16 kbps	16.23	15.77x	0.961	27.49	2.468
MP3-12k	12 kbps	8.18	31.29x	0.851	36.11	3.715
MP3-8k	8 kbps	8.18	31.29x	0.865	37.88	3.979
MP3-4k	4 kbps	8.18	31.29x	0.865	37.88	3.979
MP3-2k	2 kbps	8.18	31.29x	0.865	37.88	3.979
MP3-default	-	24.29	10.54x	0.987	22.07	1.777
AAC-16k	16 kbps	17.27	14.82x	0.967	30.73	2.881
AAC-12k	12 kbps	13.29	19.27x	0.906	33.60	3.357
AAC-8k	8 kbps	10.98	23.33x	0.870	33.76	3.464
AAC-4k	4 kbps	10.82	23.67x	0.867	33.78	3.471
AAC-2k	2 kbps	10.67	24.01x	0.866	33.79	3.475
AAC-default	-	74.99	3.42x	1.000	2.94	0.179
FLAC	-	145.88	1.76x	1.000	0.00	0.000

表 9 Full benchmark summary. This appendix preserves the entire tested operating-point set for traceability.

5 References

- [1] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, “Neural Discrete Representation Learning,” pp. 6306–6315, 2017.
- [2] K. Shen et al., “NaturalSpeech 2: Latent Diffusion Models are Natural and Zero-Shot Speech and Singing Synthesizers.” 2023.
- [3] N. Zeghidour, A. Luebs, A. Omran, J. Skoglund, and M. Tagliasacchi, “SoundStream: An End-to-End Neural Audio Codec.” 2021.
- [4] M. Tagliasacchi, Y. Li, K. Misiunas, and D. Roblek, “SEANet: A Multi-modal Speech Enhancement Network.” 2020.
- [5] A. D’efossez, J. Copet, G. Synnaeve, and Y. Adi, “High Fidelity Neural Audio Compression,” ArXiv, 2022.
- [6] K. Kumar et al., “MelGAN: Generative Adversarial Networks for Conditional Waveform Synthesis.” 2019.